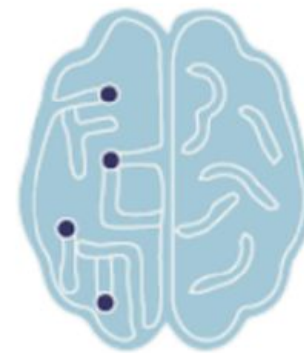


2024 FOM 미니 프로젝트 최종 발표



FOM
Focus On Data Mining

INDEX

1. 진행상황 요약
2. 모델 / 성능평가 - 홍다임
3. 모델 / 성능평가 - 오세리
4. 모델 / 성능평가 - 조혜인
5. 모델 / 성능평가 - 장하민
6. 인사이트 및 느낀점

1 | 진행상황 요약

자동차 가격 예측

- 훈련 데이터 특성들에 대한 도메인 파악
- 데이터들의 분포 / 상관관계 시각화를 통한 데이터 파악
- 필요 특성들에 대한 스케일링 및 인코딩

2 | 모델 / 성능평가 - 홍다임

자동차 가격 예측

원-핫 인코딩

: 각 카테고리를 독립된 이진열로 변환하여 해당 카테고리에만 1을 할당하고 나머지에는 0을 할당하는 방식

-> 범주형 데이터를 모델이 처리할 수 있는 숫자형 데이터로 변환하기 위해 사용

```
#원-핫 인코딩

one_hot_encoded_data = pd.get_dummies(train, columns=['브랜드', '생산년도', '차량모델명', '판매도시', '가격'])

# 결과 확인
print(one_hot_encoded_data)

# 원 핫 인코딩 결과를 새로운 CSV 파일로 저장
one_hot_encoded_data.to_csv("one_hot_encoded_data.csv", index=False)
```

	ID	모델출시년도	판매구역	주행거리	배기량	압축천연가스(CNG)	경유	가솔린	하이브리드	액화석유가스(LPG)	가격_155.35	가격_155.4	가격_155.48	가격_155.61	가격_155.74
0	TRAIN_00000	2014	SLA	85231	999	0	0	1	0	0	False	False	False	False	False
1	TRAIN_00001	2006	SWI	135000	1598	0	0	1	0	0	False	False	False	False	False
2	TRAIN_00002	2002	WIE	255223	1796	0	0	1	0	0	False	False	False	False	False
3	TRAIN_00003	2001	WIE	238000	2184	0	1	0	0	0	False	False	False	False	False
4	TRAIN_00004	2007	MAL	251000	1910	0	1	0	0	0	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
57915	TRAIN_57915	2011	WIE	64839	1461	0	1	0	0	0	False	False	False	False	False
57916	TRAIN_57916	2005	ZAC	299850	1796	0	0	1	0	0	False	False	False	False	False
57917	TRAIN_57917	2003	MAZ	112000	1368	0	0	1	0	0	False	False	False	False	False
57918	TRAIN_57918	2004	DOL	189321	1598	0	0	1	0	0	False	False	False	False	False
57919	TRAIN_57919	2004	SLA	144744	1596	0	0	1	0	0	False	False	False	False	False

2 | 모델 / 성능평가 - 홍다임

자동차 가격 예측

Ram 크기 확장

사용 가능한 RAM을 모두 사용한 후 세션이 다운되었습니다.

-> 강제 종료 문제 발생

```
from tqdm import tqdm_notebook
ram=[]
for i in tqdm_notebook(range(100000)):
    ram.append("램터트리기"*i)
```

<ipython-input-4-b1cc09e02874>:3: TqdmDeprecationWarning: This function will be removed in tqdm==5.0.0
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm\_notebook`
for i in tqdm_notebook(range(100000)):

100% 100000/100000 [00:27<00:00, 1854.19it/s]

-> 해결 완료

2 | 모델 / 성능평가 - 홍다임

자동차 가격 예측

데이터 삭제

: '가격', 'ID' 데이터 삭제

데이터 분할

: train 데이터 : test 데이터 = 8 : 2

```
# 독립 변수와 종속 변수 설정
X = train.drop(['가격', 'ID'], axis=1, errors='ignore') # 'ID'가 없어도 무시
y = train['가격']

# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```


2 | 모델 / 성능평가 - 홍다임

자동차 가격 예측

Random Forest Regressor 모델

: 여러 개의 결정 트리를 결합하여 예측하는 방식으로, 개별 트리의 약점을 상쇄하고 더 안정적인 예측을 만들어 과적합을 방지하면서도 높은 정확도 유지

```
# 모델 학습
model = RandomForestRegressor(random_state=0)
model.fit(X_train, y_train)

# 예측 및 평가
preds_train = model.predict(X_test)
print("RMSE :", np.sqrt(mean_squared_error(y_test, preds_train)))
print("R2 :", r2_score(y_test, preds_train))
```

```
RMSE : 9.667537893347948
R2 : 0.925781354177768
```

2 | 모델 / 성능평가 - 홍다임

자동차 가격 예측

수요예측 후 최종 결과 파일 저장

```
import pandas as pd

# sample_submission 파일 불러오기
submission_data = pd.read_csv('/content/drive/MyDrive/sample_submission.csv')

# 예측 결과를 sample_submission에 저장
submission_data['가격'] = preds_train

# 최종 결과 파일 저장
submission_data.to_csv('submission.csv', index=False)

from google.colab import files

# 결과 파일 다운로드
files.download('submission.csv')
```


2 | 모델 / 성능평가 - 홍다임

자동차 가격 예측

결론

Dacon 제출 결과

-> Random Forest Regressor 모델 사용

8.06372

3 | 데이터 전처리 - 오세리

자동차 가격 예측



DAICON.YT 2023.06.20 09:18

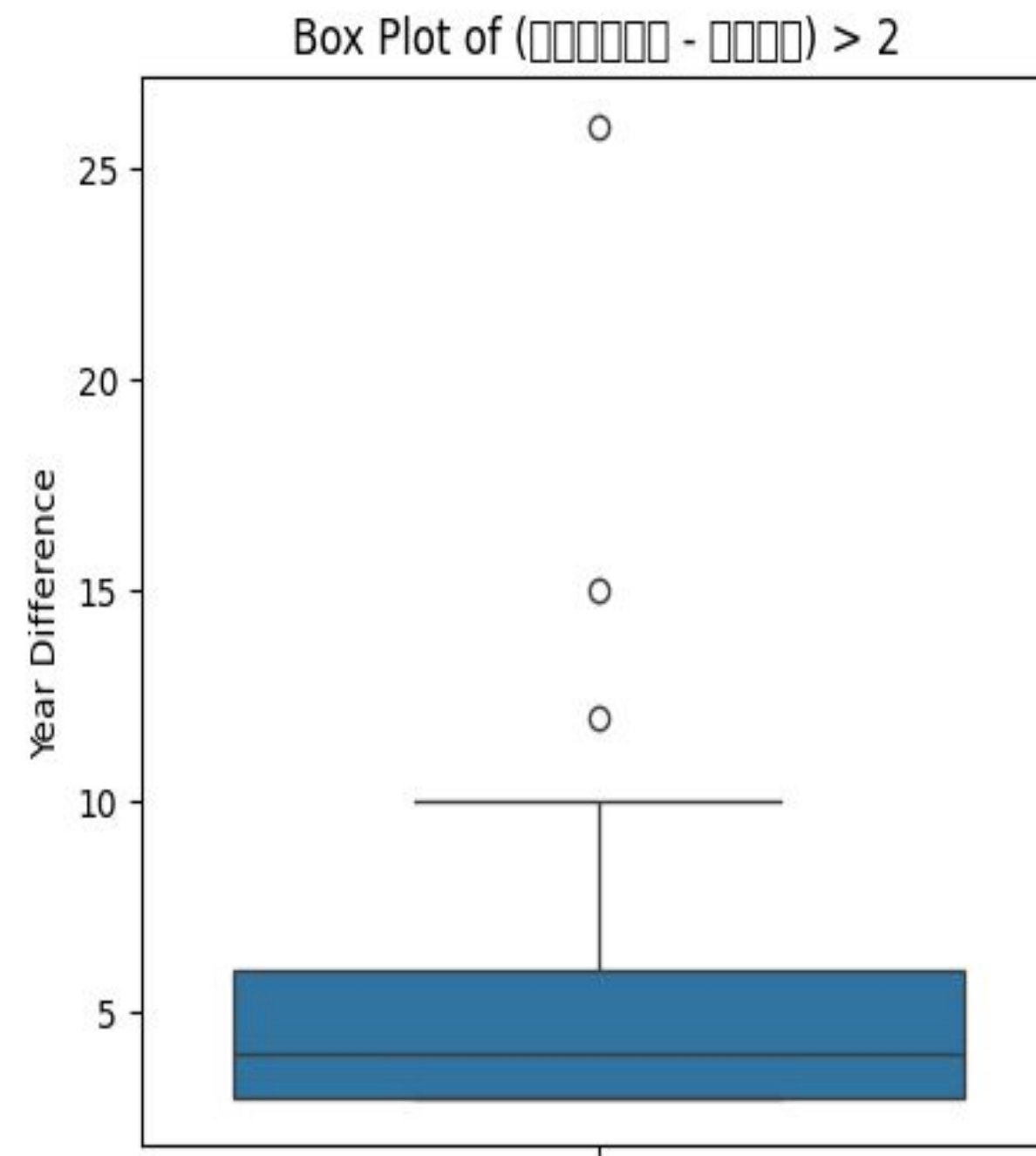
jiko님 안녕하세요.

말씀하신대로 1,2년 정도 차이나는 경우는 모델 출시를 위해 '모델출시년도' 보다 1~2년 정도 빠르게 생산되는 경우가 있습니다.

그 이상인 경우는 '모델출시년도'와 '생산년도'의 혼동으로 인한 데이터 오기입의 가능성이 있습니다.

감사합니다.

```
이상치 개수: 3
이상치 데이터:
7912      26
30450     12
36043     15
dtype: int64
```



3 | 데이터 전처리 - 오세리

자동차 가격 예측

```
def modelNmake(data):  
    data = data[(data['모델출시년도'] - data['생산년도']) < 12]  
  
    data.loc[(data['모델출시년도'] - data['생산년도']) > 2, '생산년도'] = data['모델출시년도'] + 2  
  
    return data
```

✓ 0.0s

- ['모델출시년도'] - ['생산년도']의 차가 12년 이상이면 해당 데이터를 삭제처리
- 그 이외의 값들은 생산년도를 (모델출시년도+2)값으로 대체

3 | 데이터 전처리 - 오세리

자동차 가격 예측

- 1977~1988년 데이터 삭제
- 모델출시년도와 생산년도 처리
- 주행거리 최댓값 삭제

#이상치처리

생산년도 1977~1988 사이 데이터(6개) 드롭

```
train = train[~((train['생산년도'] >= 1977) & (train['생산년도'] <= 1988))]
```

#모델출시년도, 생산년도 처리

```
train = modelNmake(train)
```

#주행거리 최댓값 삭제

```
max_mileage = train['주행거리'].max()
```

```
train = train[train['주행거리'] != max_mileage]
```

✓ 0.0s

3 | 데이터 전처리 - 오세리

자동차 가격 예측

Stand Scaling(수치형 데이터)

- 주행거리의 이상치 수 : 481 -1
- 배기량 의 이상치 수 : 3528
- 이상치에 덜 민감
- 데이터 분포의 형태를 유지

```
scaler = StandardScaler()

# 주행거리 열 정규화
train['주행거리'] = scaler.fit_transform(train[['주행거리']])

# 배기량 열 정규화
train['배기량'] = scaler.fit_transform(train[['배기량']])
```

✓ 0.0s

3 | 데이터 전처리 - 오세리

자동차 가격 예측

범주형 데이터 라벨인코딩

- 범주를 고유의 숫자 값으로 매핑
- 브랜드, 차량모델명, 판매도시, 판매구역
라벨인코딩

```
#라벨인코딩
ordinal_features = ['브랜드', '차량모델명', '판매도시', '판매구역']

for feature in ordinal_features:
    le = LabelEncoder()
    le = le.fit(train[feature])
    train[feature] = le.transform(train[feature])
    |
    for label in np.unique(test[feature]):
        |
        |
        | if label not in le.classes_:
        |     |
        |     | le.classes_ = np.append(le.classes_, label)
        |     |
        test[feature] = le.transform(test[feature])
```

✓ 0.2s

3 | 데이터 전처리 - 오세리

자동차 가격 예측

모델링 준비

- 모델링에 필요하지 않은 ID와 가격을 `train_x` 데이터에서 제외
- `train_y` 데이터에 가격 대입

```
#ID와 가격 드롭
print(train.columns)
train_x = train.drop(['ID', '가격'], axis = 1)
train_y = train['가격']

test_x = test.drop('ID', axis = 1)
print(train_x.columns)
print(train_x.info())

#
train_y = train['가격']
```

3

데이터 전처리 - 오세리

자동차 가격 예측

```
# train 데이터를 train과 validation으로 8:2로 분할

# train_x와 train_y를 훈련 데이터와 검증 데이터로 나누기
train_x, validation_x, train_y, validation_y = train_test_split(train_x, train_y, test_size=0.2, random_state=42)

print(train_x.shape, train_y.shape)
print(validation_x.shape, validation_y.shape)
```

✓ 0.0s

train 데이터와 validation 데이터를 각각 80%와 20%로 나눔

3

모델 / 성능평가 - 오세리

자동차 가격 예측

XGBRegrssor

```
#회귀모델 정의, 모델학습
dtrain= xgb.DMatrix(train_x, label=train_y)
dval = xgb.DMatrix(validation_x, label=validation_y)

# XGBoost 모델 하이퍼파라미터 설정
params = {
    'objective': 'reg:squarederror', # 회귀 문제일 경우
    'eval_metric': 'mae', # MAE 사용
    'eta': 0.1, # 학습률
    'max_depth': 6, # 트리 깊이
    'subsample': 0.8, # 데이터 샘플 비율
    'colsample_bytree': 0.8 # 트리에서 사용하는 열 비율
}

num_round = 100 # 훈련 반복 횟수

# 모델 훈련
model = xgb.train(params, dtrain, num_round, evals=[(dval, 'eval')], early_stopping_rounds=10)

# 예측
predictions = model.predict(dval)

# 평가: MAE 계산
mae = mean_absolute_error(validation_y, predictions)
print(f'Mean Absolute Error (MAE): {mae}')

✓ 0.8s
```

Mean Absolute Error (MAE): 6.5963301874081886

DecirionTreeRegressor

```
model = DecisionTreeRegressor(random_state = 42)
model.fit(train_x, train_y)
y_pred = model.predict(validation_x)
# MSE 계산
mse = mean_absolute_error(validation_y, y_pred)

print(f"Mean Squared Error (MSE): {mse}")
```

✓ 0.4s

Mean Squared Error (MSE): 8.613882438549105

Mean Squared Error (MSE): 182.5770126430836

3

모델 / 성능평가 - 오세리

자동차 가격 예측

LGBMRegrssor

```
model_L = LGBMRegressor(max_depth=5, learning_rate=0.1, n_estimators=3000, random_state=42)
model_L.fit(train_x, train_y)
```

```
# 예측 및 평가
y_pred_L = model.predict(validation_x) # validation_x를 사용하여 예측
mae = mean_absolute_error(validation_y, y_pred_L)
print(f'Mean Absolute Error (MAE): {mae:.2f}')
```

✓ 8.5s

Pytho

```
[LightGBM] [Warning] No further s
Mean Absolute Error (MAE): 5.92
```

CatBoostRegrssor

```
model = CatBoostRegressor(iterations=4000, # 트리의 수
                           learning_rate=0.01, # 학습률
                           depth=10, # 트리 깊이
                           loss_function='MAE', # 손실 함수 (Mean Absolute Error)
                           cat_features=[]
                           )
```

```
# 모델 학습
model.fit(train_x, train_y,
          eval_set=(validation_x, validation_y), # 검증 데이터
          early_stopping_rounds=10, # 10번의 라운드 동안 성능 향상이 없다면 학습 종료
          verbose=100)
```

```
# 예측
y_pred = model.predict(validation_x)
```

```
# 평가 (MAE)
mae = mean_absolute_error(validation_y, y_pred)
print(f'Mean Absolute Error: {mae}')
```

Mean Absolute Error: 5.942268273012712

3 | 모델 / 성능평가 - 오세리

자동차 가격 예측

CatBoostRegrssor + XGBRegrssor
-> VotingRegressor ensemble

```
catboost_model = CatBoostRegressor(iterations=3000, learning_rate=0.05, depth=9, verbose=0)
xgboost_model = XGBRegressor(n_estimators=3000, learning_rate=0.05, max_depth=9)

ensemble_model = VotingRegressor(estimators=[('catboost', catboost_model), ('xgboost', xgboost_model)])

ensemble_model.fit(train_x, train_y)

y_pred = ensemble_model.predict(validation_x)

mae = mean_absolute_error(validation_y, y_pred)
print(f'Mean Absolute Error: {mae}')
```

✓ 54.0s

Mean Absolute Error: 5.758783500346349

3 | 모델 / 성능평가 - 오세리

자동차 가격 예측

결과

점수

CatBoostRegrssor + XGBRegrssor-> VotingRegressor ensemble

41.26732

54.7292225697 LGBMRegrssor

-> 모델학습과정에서 과적합이 발생한 것으로 추정된다.

4 | 모델 / 성능평가 - 조혜인

자동차 가격 예측

1. 이상치

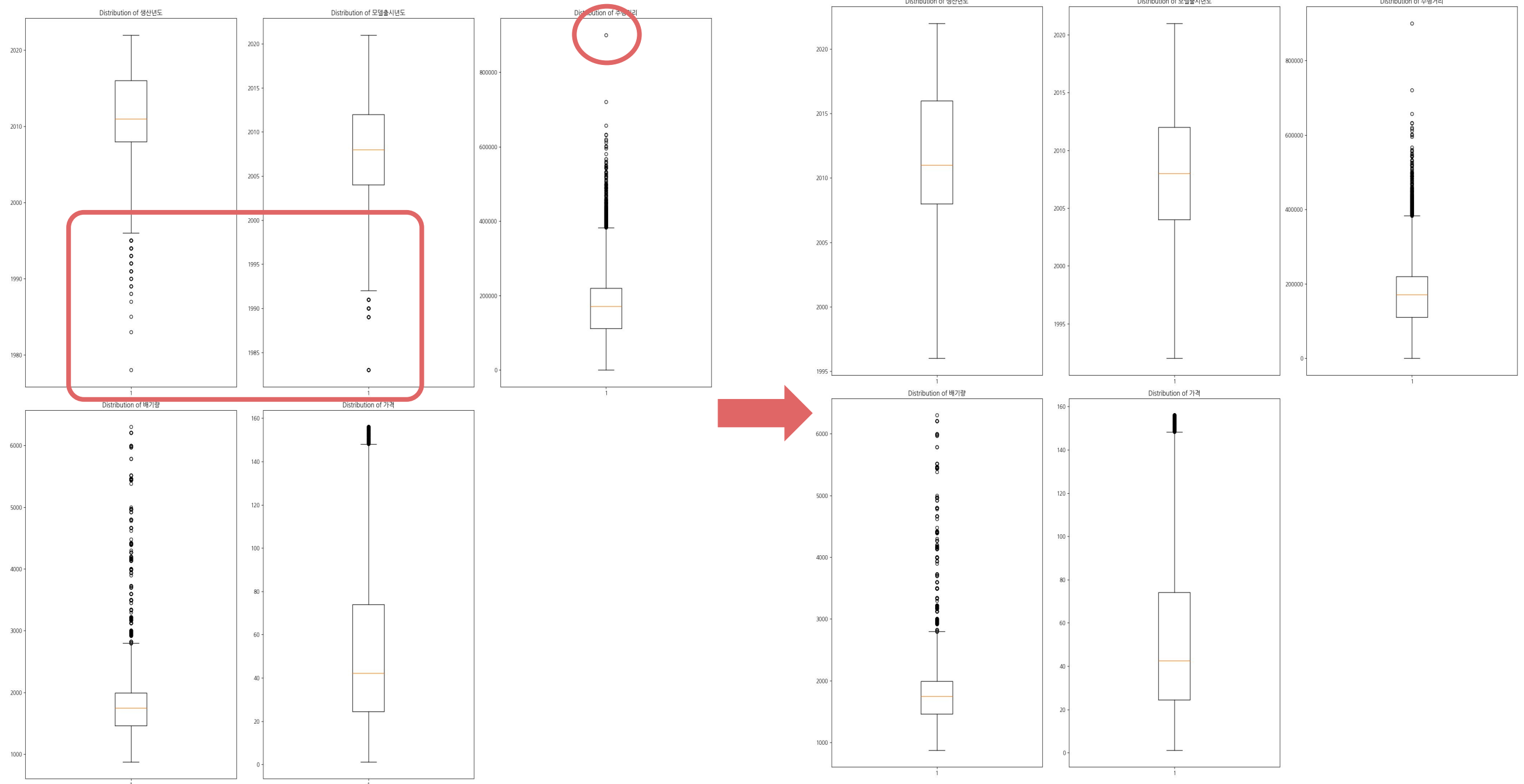
- 유독 큰 주행거리 데이터 1개 제거

```
train = train.drop(index = 'TRAIN_40808')
```

- ‘생산년도’, ‘모델출시년도’가 너무 오래된 데이터 제거

```
def remove_outlier(df, col):  
    Q1 = df[col].quantile(0.25)  
    Q3 = df[col].quantile(0.75)  
    IQR = Q3 - Q1  
    lower_bound = Q1 - 1.5 * IQR  
    upper_bound = Q3 + 1.5 * IQR  
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]  
    return df
```

```
train = remove_outlier(train, '생산년도')  
train = remove_outlier(train, '모델출시년도')
```



4 | 모델 / 성능평가 - 조혜인

자동차 가격 예측

2. 연료 타입 변환

- 5개의 연료를 1개의 컬럼으로 변환. (복잡성 down)
- ['압축천연가스(CNG)', '경유', '가솔린', '하이브리드', '액화석유가스(LPG)'] 컬럼 제거

3. '연식' 컬럼 추가 및 '생산년도' 제거

```
def preprocess_data(df):  
    # 연료 타입을 하나의 컬럼으로 합치는 작업  
    df['연료'] = df.apply(get_fuel_type, axis=1)  
  
    # 불필요한 컬럼 제거  
    df = df.drop(['압축천연가스(CNG)', '경유', '가솔린', '하이브리드', '액화석유가스(LPG)'], axis=1)  
  
    ## 추가적인 전처리 작업  
    # 연식 컬럼 추가  
    df['연식'] = 2022 - df['생산년도']  
  
    # 생산년도 컬럼 제거  
    df = df.drop(['생산년도'], axis=1)
```

columns = ['모델출시년도', '브랜드', '차량모델명', '판매도시', '판매구역', '주행거리', '배기량', '연료', '연식']

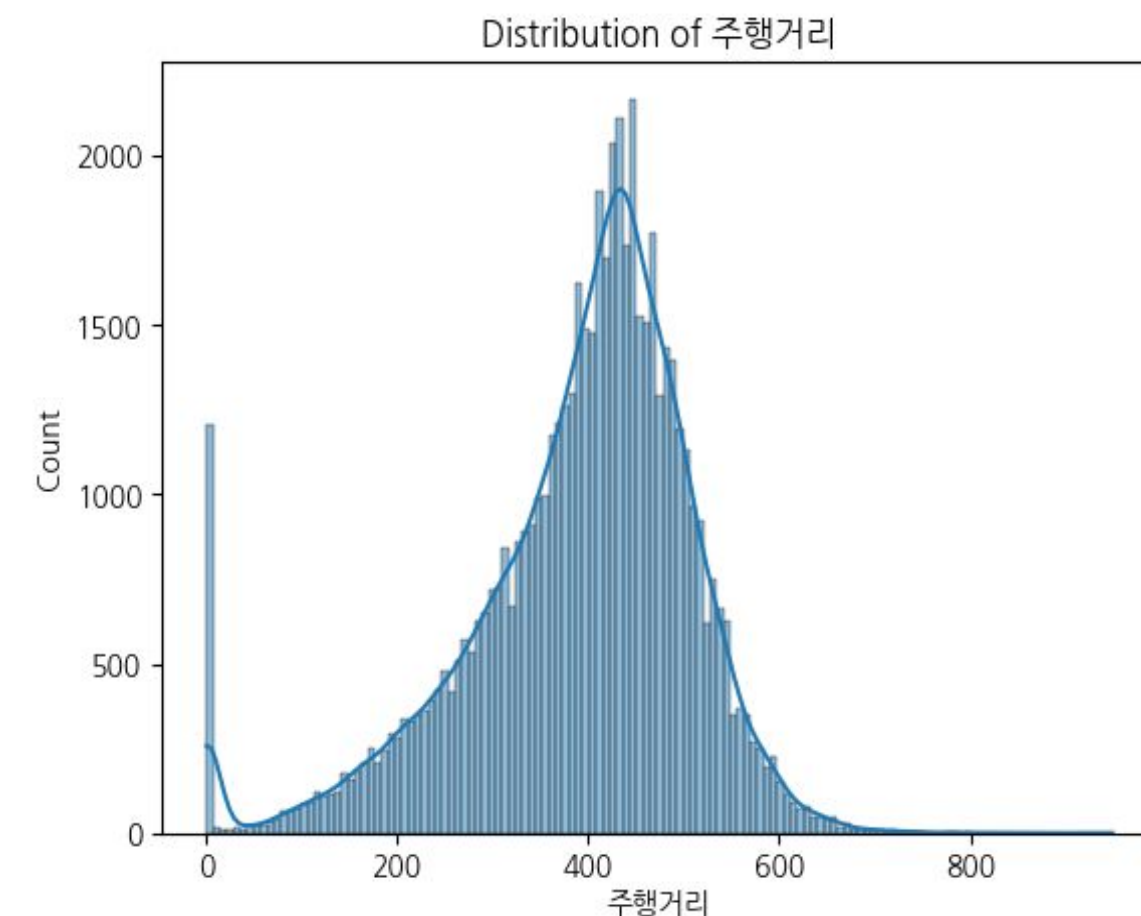
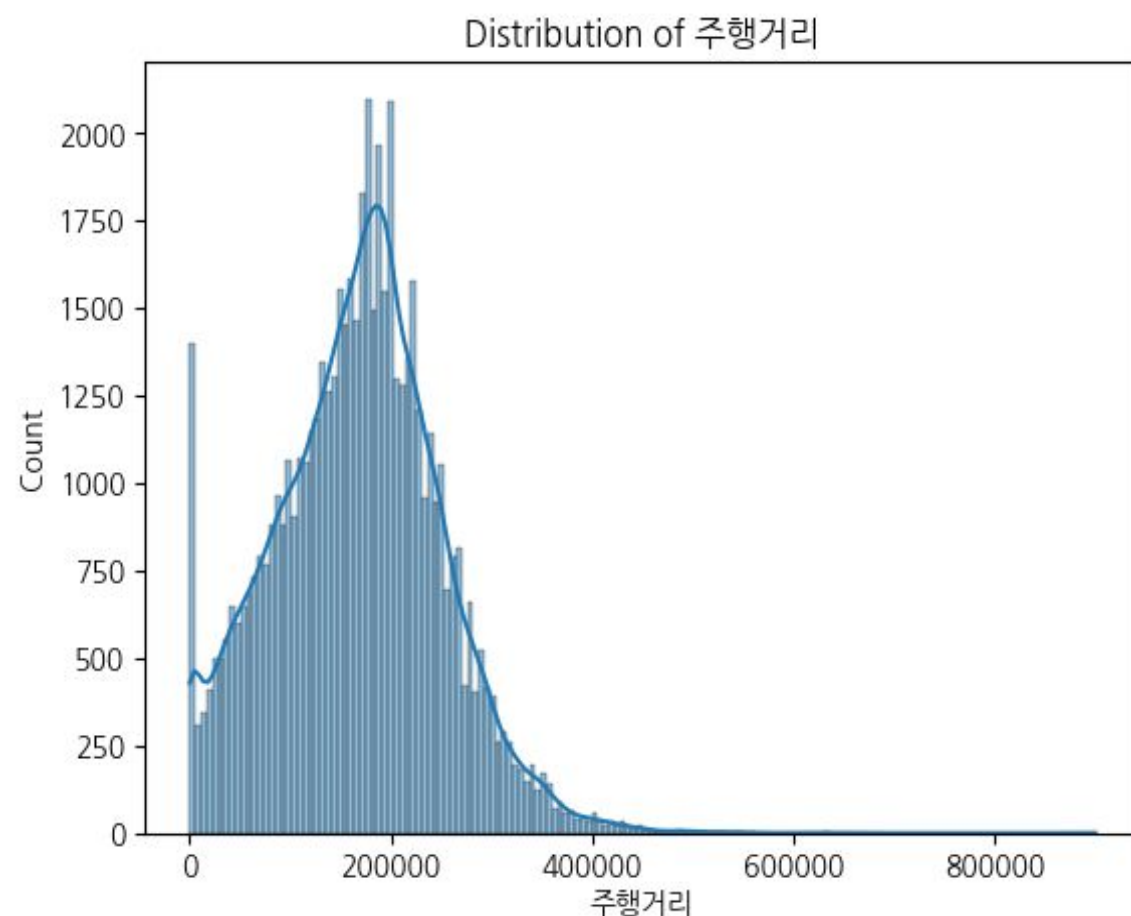
4 | 모델 / 성능평가 - 조혜인

자동차 가격 예측

4. 변환 : 로그변환 => 제곱근 변환

- '주행거리' 제곱근 변환
 - 주행거리가 0인 값 존재
 - log는 0에서 정의되지 않으므로, 제곱근 변환으로 변경

```
# 제곱근변환  
df['주행거리'] = np.sqrt(df['주행거리'])
```



4 | 모델 / 성능평가 - 조혜인

자동차 가격 예측

5. encoding

one-hot encoding : 변수의 수를 급격하게 증가시킬 수 있음.

label encoding : 숫자로 변형되어 순서 또는 등급 부여됨.

catboost regressor은 범주형 데이터를 자동으로 처리하는 특징

=>encoding을 따로 진행해주지 않았음.

	모델출시년도	브랜드	차량모델명	판매도시	판매구역	주행거리	배기량	가격	연료	연식
ID										
TRAIN_00000	2014	skoda	fabia	KAT	SLA	291.943488	999	51.74	가솔린	4
TRAIN_00001	2006	toyota	auris	RKO	SWI	367.423461	1598	41.47	가솔린	12
TRAIN_00002	2002	mercedes-benz	clk-klasa	GNI	WIE	505.196002	1796	17.81	가솔린	20
TRAIN_00003	2001	nissan	x-trail	EHX	WIE	487.852437	2184	18.20	경유	16
TRAIN_00004	2007	fiat	bravo	OSW	MAL	500.999002	1910	17.55	경유	15
...	...	...	...	...	...	...	...	...	...	...
TRAIN_57915	2011	mercedes-benz	b-klasa	POZ	WIE	254.635033	1461	116.87	경유	5
TRAIN_57916	2005	opel	zafira	XOK	ZAC	547.585610	1796	15.47	가솔린	16
TRAIN_57917	2003	fiat	punto	CIE	MAZ	334.664011	1368	21.97	가솔린	9
TRAIN_57918	2004	opel	astra	ZRI	DOL	435.110331	1598	16.77	가솔린	16
TRAIN_57919	2004	bmw	seria-1	EIX	SLA	380.452362	1596	19.50	가솔린	17

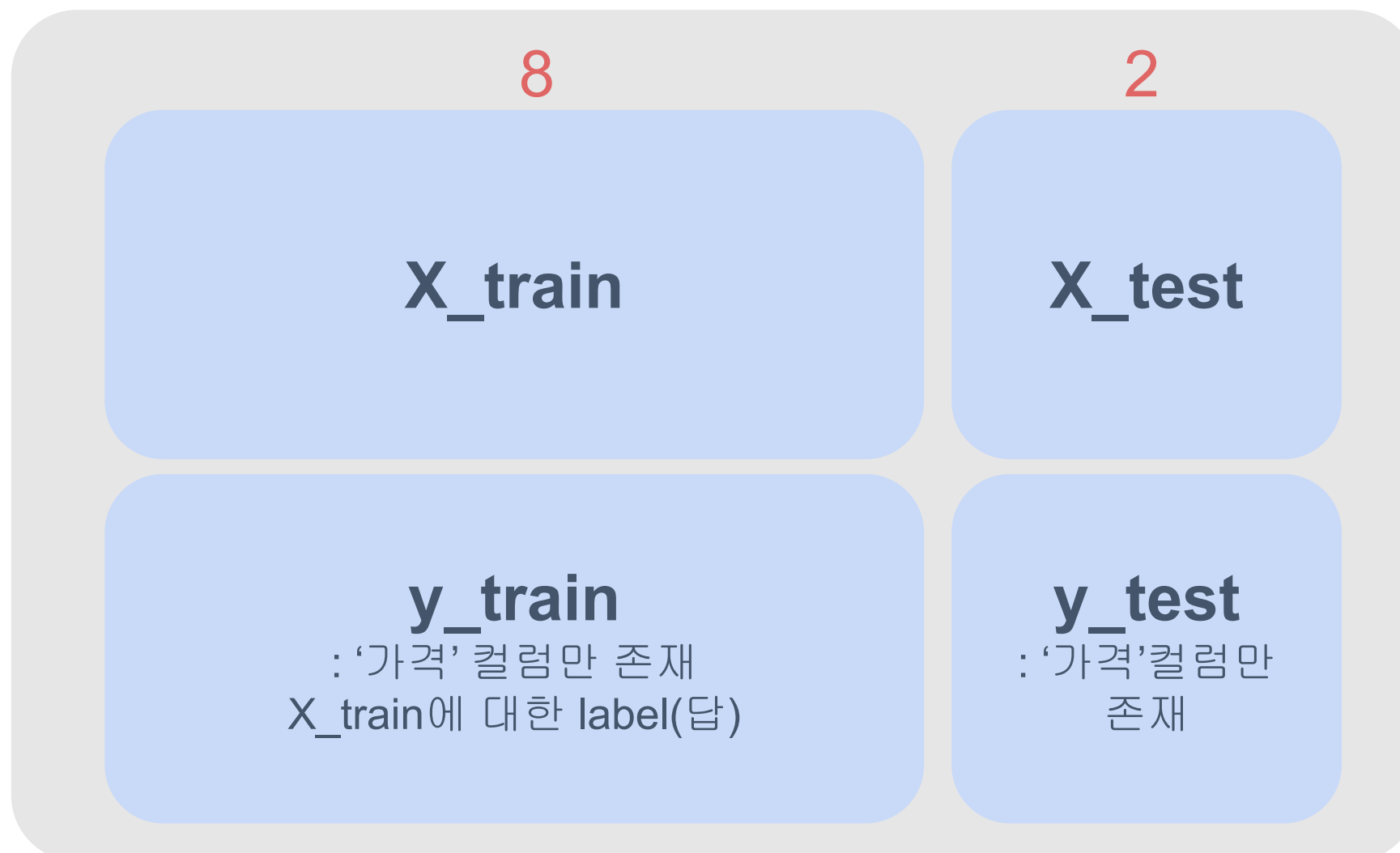
4 | 데이터 전처리 / 시각화 - 조혜인

데이터 셋 분리

6. 데이터셋 분리하기

```
import sklearn.model_selection as model_selection  
  
X_train, X_test, y_train, y_test = model_selection.train_test_split(train.drop('가격',axis=1), train['가격'], test_size=0.2, random_state=42)
```

train 데이터



- X_train, y_train : 모델 훈련
- X_test, y_test : 성능 측정
- X_train, y_train으로 모델 훈련
- X_test으로 predict함. (y_pred)
- y_pred와 y_test의 값을 비교하여 오차 확인

4 | 모델 / 성능평가 - 조혜인

자동차 가격 예측

7. 모델 : CatBoost Regressor

- Boosting 기반 모델 (약한 분류기의 결합으로 강한 분류기 생성)
- Categorical Boosting : 범주형 변수를 자동으로 처리함.
- 하이퍼파라미터를 자동으로 조절하는 기능이 내장됨.
- overfitting 적음.
- 결측치 많은 데이터에서는 부적합.

```
from sklearn.metrics import mean_absolute_error
from catboost import CatBoostRegressor

ml_cat = CatBoostRegressor(eval_metric='MAE', random_state=42)
cat_features = ['브랜드', '차량모델명', '판매도시', '판매구역', '연료']

ml_cat.fit(X_train, y_train, cat_features=cat_features)
```

4 | 모델 / 성능평가 - 조헤인

자동차 가격 예측

8. 하이퍼파라미터 튜닝

Grid Search

- 사이킷런(Scikit-learn) 라이브러리에서 제공하는 하이퍼파라미터 튜닝 기법
- 지정된 하이퍼파라미터의 모든 조합을 시도하여 **최적의 파라미터 조합**을 찾는 과정
- 많은 시간 소요, 계산비용 多

scoring : 각 하이퍼파라미터 조합을 모델평가에 사용되는 성능 지표.

cv : cross validation, 교차검증을 위해 분할되는 수.

4 | 모델 / 성능평가 - 조혜인

자동차 가격 예측

9. 결과

```
ml_cat = CatBoostRegressor(eval_metric='MAE',
                             random_state=42,
                             depth=10,
                             iterations=400,
                             l2_leaf_reg=1,
                             learning_rate=0.1)

ml_cat.fit(X_train, y_train, cat_features=cat_features)
y_cat = ml_cat.predict(X_test)
mae = mean_absolute_error(y_test, y_cat)
print("cat ver MAE:", mae)
```

```
390:   learn: 4.7541452      total: 58.3s   remaining: 1.34s
391:   learn: 4.7501889      total: 58.6s   remaining: 1.2s
392:   learn: 4.7488163      total: 58.8s   remaining: 1.05s
393:   learn: 4.7473867      total: 59s     remaining: 898ms
394:   learn: 4.7466142      total: 59.2s   remaining: 749ms
395:   learn: 4.7433018      total: 59.4s   remaining: 600ms
396:   learn: 4.7423434      total: 59.7s   remaining: 451ms
397:   learn: 4.7374547      total: 59.9s   remaining: 301ms
398:   learn: 4.7323319      total: 1m      remaining: 151ms
399:   learn: 4.7307379      total: 1m      remaining: 0us

cat ver MAE: 5.806462390153629
```

```
제안된 코드에 라이선스가 적용될 수 있습니다.
1 y_cat= ml_cat.predict(test)
2 submission['가격'] = y_cat
3 submission.to_csv('submit_cat_ver_2.csv')
4 submission
```

가격	
ID	
TEST_00000	89.120088
TEST_00001	26.718514
TEST_00002	84.090881
TEST_00003	126.244889
TEST_00004	47.102962
...	
TEST_14475	41.262396
TEST_14476	23.551333
TEST_14477	109.061774
TEST_14478	50.163376
TEST_14479	44.401065

14480 rows × 1 columns

점수

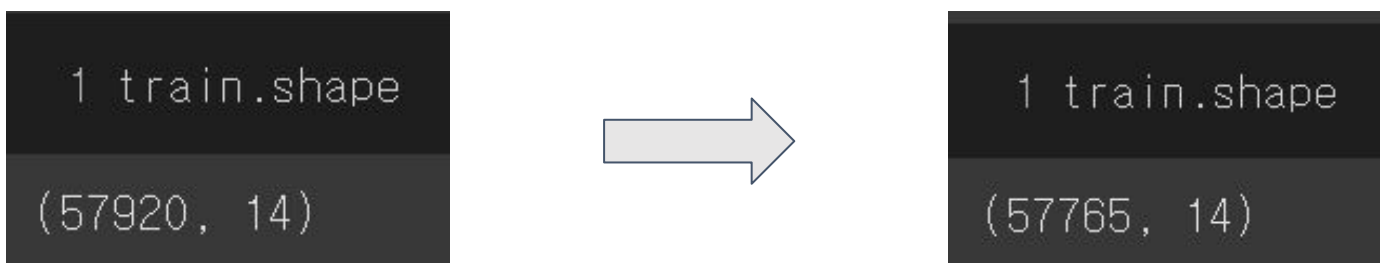
5.88379

- local mae
-> 5.806
- Dacon 제출 ver mae
-> 5.883

5 | 모델 / 성능평가 - 장하민

자동차 가격 예측

- 전처리 - 진행상황
 - 중복값 처리, 주행거리, 배기량 극단값들 이상치 처리
 - 생성한 연도 변수의 27년식 이상의 차들 이상치 처리
 - catboost 사용 - 인코딩 진행 x
- 추가적으로 각 특성별로 이상치들 처리하며 모델 성능 확인 후 유의미한 값들 제거



5

모델 / 성능평가 - 장하민

자동차 가격 예측

- **모델 : CatBoostRegrssor**

- 범주형 데이터에 특화된 그래디언트 부스팅기반 알고리즘
- 오버피팅 방지 / 높은 성능 / 적은 하이퍼파라미터 튜닝
- 범주형 데이터 자동 처리 : 범주형 데이터를 직접 처리할 수 있어서, 원-핫 인코딩이나 레이블 인코딩같은 전처리가 필요 없고 내부에서 범주형 데이터를 효과적으로 처리하기 위한 인코딩 진행(타겟 값의 평균을 반영)

5

모델 / 성능평가 - 장하민

자동차 가격 예측

- 하이퍼 파라미터

- 그리드 서치 : 지정한 하이퍼 파라미터의 모든 조합을 탐색

- 모든 하이퍼 파라미터를 탐색하므로 최적의 하이퍼 파라미터를 찾을 가능성이 높지만 범위가 클 경우 시간이 매우 많이 소모됨

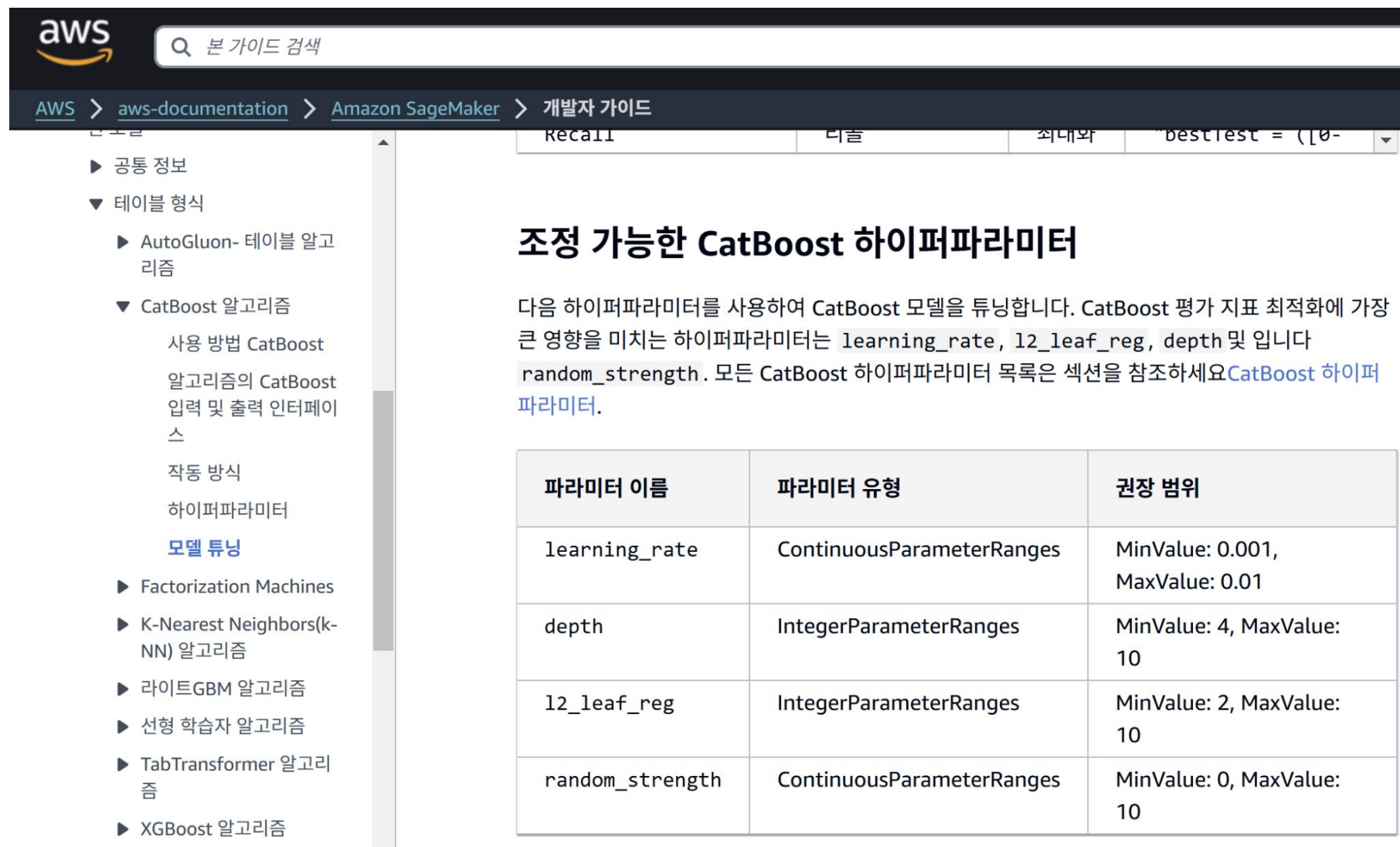
- 랜덤 서치: 설정한 하이퍼 파라미터의 조합 중 임의로 선택해서 탐색

- 모든 조합을 시도하지 않고, 선택된 횟수만큼 탐색하여 시간을 단축시킴

5 | 모델 / 성능평가 - 장하민

자동차 가격 예측

● 하이퍼 파라미터



The screenshot shows the AWS SageMaker documentation page for CatBoost hyperparameters. The left sidebar contains a navigation menu with categories like '공통 정보', '테이블 형식', 'CatBoost 알고리즘', '작동 방식', '하이퍼파라미터', '모델 튜닝', and various machine learning algorithms. The main content area is titled '조정 가능한 CatBoost 하이퍼파라미터' (Adjustable CatBoost Hyperparameters). It includes a paragraph explaining that the following hyperparameters are used for CatBoost model tuning: learning_rate, l2_leaf_reg, depth, and random_strength. Below this text is a table listing these hyperparameters with their types and recommended ranges.

파라미터 이름	파라미터 유형	권장 범위
learning_rate	ContinuousParameterRanges	MinValue: 0.001, MaxValue: 0.01
depth	IntegerParameterRanges	MinValue: 4, MaxValue: 10
l2_leaf_reg	IntegerParameterRanges	MinValue: 2, MaxValue: 10
random_strength	ContinuousParameterRanges	MinValue: 0, MaxValue: 10

```
5 param_grid = {  
6     'iterations' : [500, 1000, 1500],  
7     'learning_rate' : [0.001, 0.005, 0.01, 0.05, 0.1],  
8     'depth' : [4, 6, 8, 10],  
9     'l2_leaf_reg' : [1, 3, 5, 7, 9],  
10    'random_strength' : [0.5, 1, 2, 5],  
11    'bagging_temperature' : [0, 0.5, 1, 2]  
12 }
```

- AWS에서 제공하는 cat boost 하이퍼 파라미터 권장범위 + 추가적인 파라미터 조합으로 랜덤 서치 진행

5 | 모델 / 성능평가 - 장하민

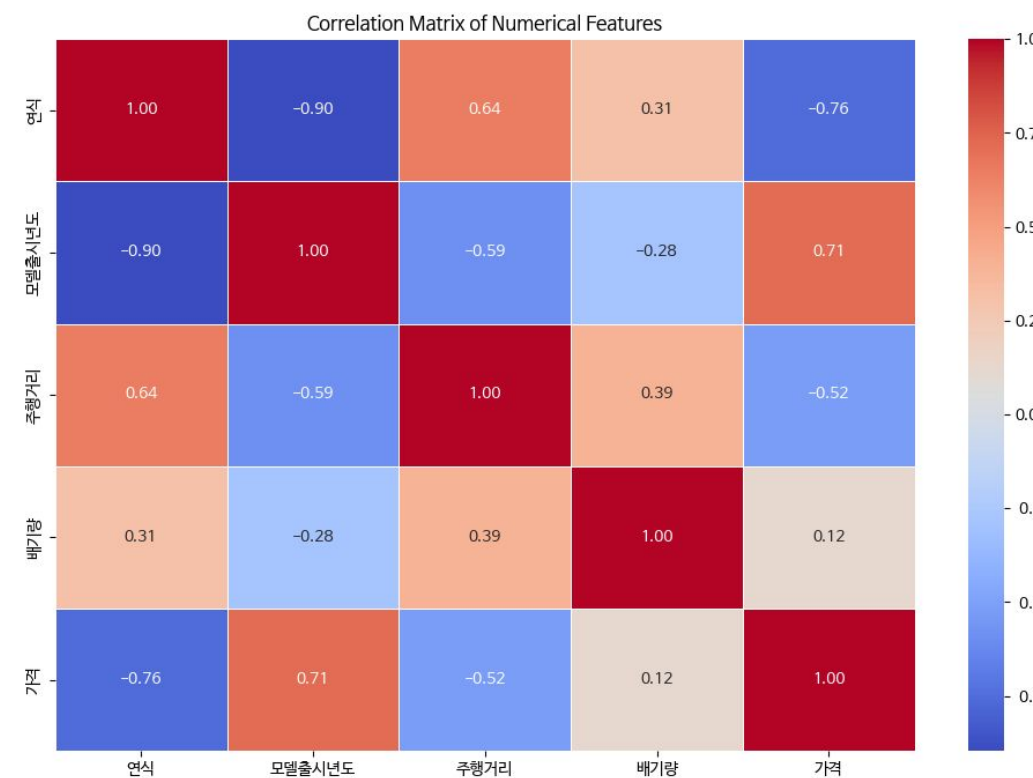
자동차 가격 예측

● 결과

```
1 print(cat1.get_feature_importance(prettified=True))
```

	Feature Id	Importances
0	연식	34.428894
1	배기량	20.244878
2	차량모델명	13.242862
3	브랜드	9.150475
4	모델출시년도	9.047774
5	주행거리	8.809147
6	판매도시	1.874212
7	판매구역	1.339888
8	경유	0.907030
9	가솔린	0.629203
10	하이브리드	0.247203
11	액화석유가스(LPG)	0.064118
12	압축천연가스(CNG)	0.014317

feature_importance



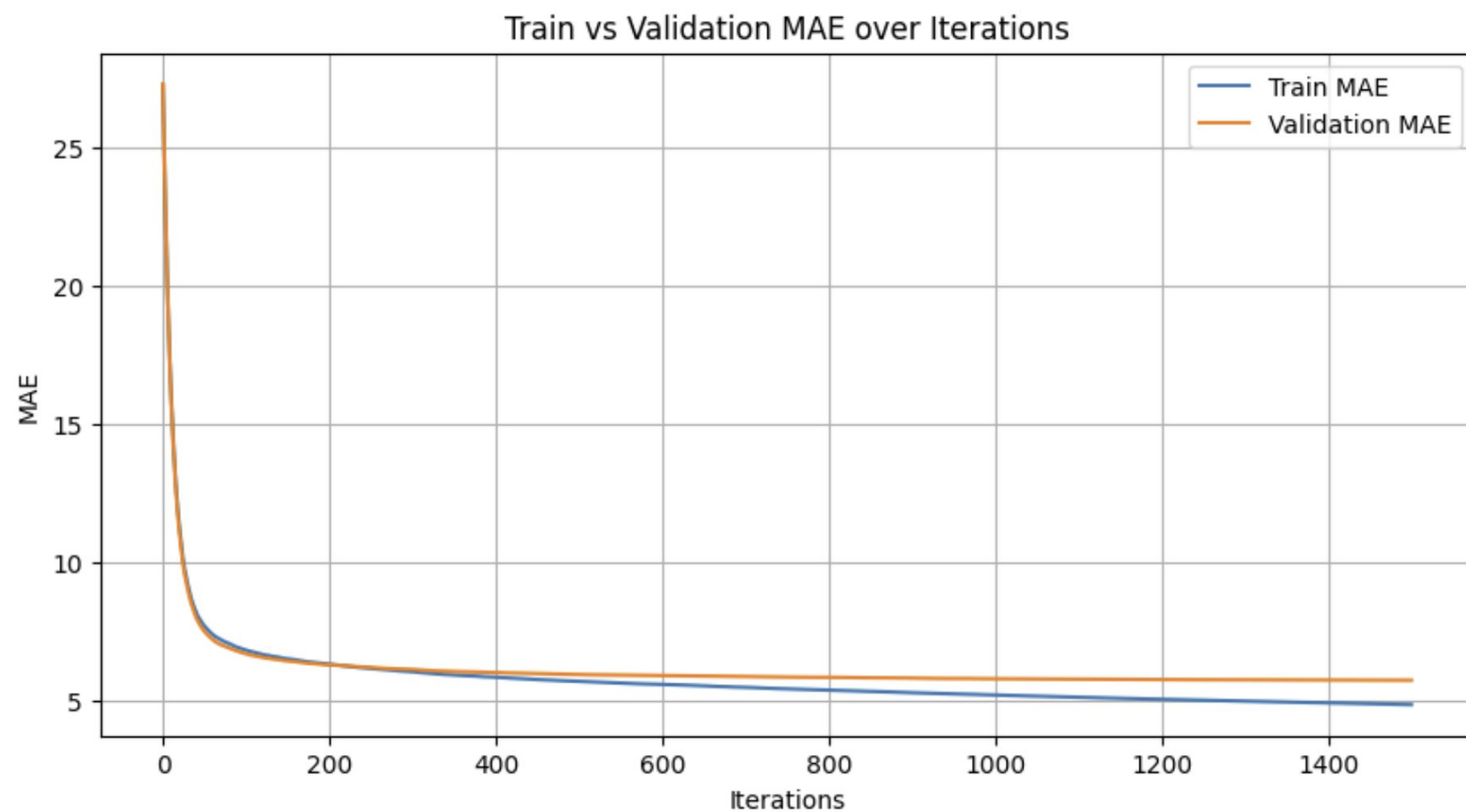
상관계수

- 상관계수는 각 변수간의 선형적인 관계를 나타냄
- feature importance는 모델(주로 트리 기반) 예측에 기여하는 정도에 따라 중요도를 나타냄
- 상관계수와 feature_importance를 상호 보완적으로 해석하며 변수가 미치는 영향을 분석

5 | 모델 / 성능평가 - 장하민

자동차 가격 예측

● 결과



```
bestTest = 5.730599042  
bestIteration = 1497
```

- Local MAE : 5.7305

- Dacon 제출 MAE : 5.786

점수

5.78608

6

최종 결론 및 인사이트

자동차 가격 예측

장하민 : 트리 기반 회귀 모델은 많이 생소했는데 범주형 데이터 처리에 알맞은 **CatBoost regressor** 모델을 발견하고 사용해 볼 수 있어서 좋은 경험이었고, 다양한 시각화와 하이퍼 파라미터 튜닝까지 전체적인 데이터 처리과정을 잘 진행해 본 것 같습니다.

조혜인 : 범주형 데이터를 어떤 방식으로 인코딩 해야할지 고민이 많았는데, 인코딩이 필요없는 **catboost** 모델이 있음을 새롭게 알 수 있는 기회였습니다. 또한, **Grid Search** 실행시간이 너무 오래 걸려서 가장 최적의 파라미터를 찾은 것이 맞는지를 알 수 없다는 점이 좀 아쉬웠습니다.

홍다임 : 데이터 분석을 처음 진행해보았는데 데이터 분석을 완벽하게 하진 못했지만, 직접해봄으로써 어떤 식으로 진행해야하는지 알 수 있었습니다. 데이터 전처리하고 시각화하는 과정이 중요하다고 느껴, 앞으로 기초적인 지식을 더 공부하고 많은 데이터를 분석해보면서 능력을 향상시키기 위해 노력할 예정입니다.

오세리 : 데이터 시각화, 전처리, 모델학습까지의 일련의 과정이 처음이어서 미흡한 점이 많다고 스스로 느껴지는 프로젝트였습니다. 때문에 아쉬움이 더 큰 마음입니다. 특히 데이터 시각화를 통하여 데이터에 대한 인사이트를 얻는 과정과, 모델선정, 하이파라미터를 조정하는 부분을 조금 더 학습하고 싶습니다.

김유진: 데이터 전처리, 시각화까지 진행하면서 기본 학습이 부족함을 느꼈고, 프로젝트 전반적으로 적극적으로 참여하지 못한 점이 아쉽습니다. 데이터 분석 과정에 대한 이해와 더 적극적인 관련 지식 학습을 통해 향후 새로운 분석과제가 있을 때는 더 개선된 결과를 낼 수 있도록 노력하고 싶습니다.



Q & A

감사합니다